

Compiladores

23 de febrero de 2026

Martínez Buenrostro Jorge Rafael

correo: cbi2203040824@izt.uam.mx

Universidad Autónoma Metropolitana

Unidad Iztapalapa, México

Introducción

El presente reporte describe la implementación de cuatro Autómatas Finitos Deterministas (AFD) diseñados para reconocer lenguajes regulares específicos. La implementación se realizó en lenguaje C, utilizando estructuras de datos para representar la lógica de estados y transiciones.

Análisis de Autómatas

1. Expresión Regular a^*ba^*

Este autómata acepta cadenas que contienen exactamente una letra 'b', precedida y seguida por cualquier cantidad de letras 'a'.

Definición Formal: $M_1 = \{Q, \Sigma, \delta, q_0, F\}$, donde:

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- $s = q_0$
- $F = \{q_1\}$

Tabla de Transiciones:

Estado	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

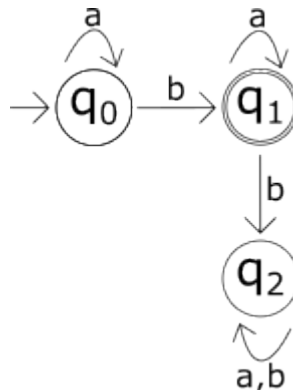


Figura 1: Diagrama del autómata para a^*ba^*

2. Expresión Regular a^*bba^*

Acepta cadenas que contienen la subcadena exacta 'bb', con cualquier cantidad de 'a' alrededor, pero sin permitir más 'b's.

Definición Formal: $M_2 = \{Q, \Sigma, \delta, q_0, F\}$, donde:

- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = \{a, b\}$
- $s = q_0$
- $F = \{q_2\}$

Tabla de Transiciones:

Estado	a	b
q_0	q_0	q_1
q_1	q_3	q_2
q_2	q_2	q_3
q_3	q_3	q_3

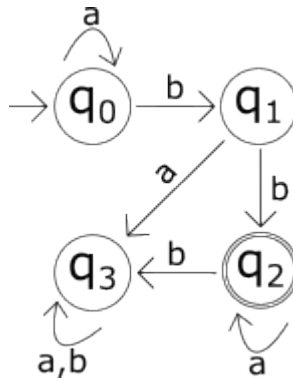


Figura 2: Diagrama del autómata para a^*bba^*

3. Expresión Regular a^*b^*

Reconoce cadenas que tienen cero o más 'a' seguidas de cero o más 'b'. Una vez que aparece una 'b', no puede volver a aparecer una 'a'.

Definición Formal: $M_3 = \{Q, \Sigma, \delta, q_0, F\}$, donde:

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- $s = q_0$
- $F = \{q_0, q_1\}$

Tabla de Transiciones:

Estado	a	b
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_2	q_2

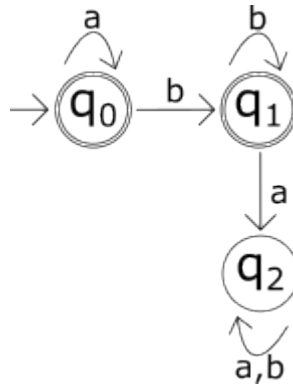


Figura 3: Diagrama del autómata para a^*b^*

4. Expresión Regular $a^*ba^*ba^*$

Este autómata acepta cadenas que contienen exactamente dos letras 'b'.

Definición Formal: $M_4 = \{Q, \Sigma, \delta, q_0, F\}$, donde:

- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = \{a, b\}$
- $s = q_0$
- $F = \{q_2\}$

Tabla de Transiciones:

Estado	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_3
q_3	q_3	q_3

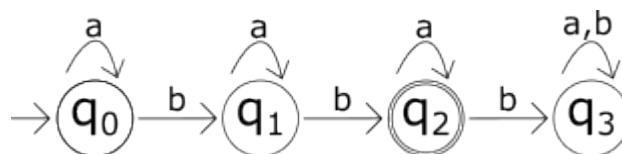


Figura 4: Diagrama del autómata para $a^*ba^*ba^*$

Código fuente

```
\#include<stdio.h>

typedef struct Cadena\{
char * wn;
int longitud;
\}Cadena;

typedef struct Automata\{
int num\_estados;
int num\_simbolos;
int estado\_inicial;
int * estados\_finales;
int num\_finales;
int** transiciones;
\}Automata;

typedef Cadena * cadena;
typedef Automata * automata;

cadena crearCadena();
void pedirCadena(cadena);
void liberarCadena(cadena);

/*
num\_estados: numero de estados
num\_simbolos: numero de simbolos del alfabeto
estado\_inicial: estado inicial
estados\_finales: arreglo con los estados finales
transiciones: matriz de transiciones
*/
automata crearAutomata(int num\_estados, int num\_simbolos, int
    estado\_inicial,
int* estados\_finales, int num\_finales, int** transiciones);
int procesarCadena(automata, cadena);
void liberarAutomata(automata);
int crearAutomatas(automata automatas[4]);

void menuExpresiones(int, automata automatas[4]);
```

Listing 1: Archivo cabecera

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include ‘‘funciones.h’’

cadena crearCadena()\{
cadena a;

a = (cadena) malloc(sizeof(Cadena));
if(a == NULL)\{
return NULL;
\}

a->wn = (char*) malloc(sizeof(char));
if(a->wn == NULL)\{
free(a);
return NULL;
\}

a->wn[0] = '\\textbackslash{}0';
a->longitud = 0;

return a;
\}

void pedirCadena(cadena a)\{
scanf(‘‘\\ %[^{}\\textbackslash{}n]s’’,a->wn);
getchar();

int longitud = 0;
while(a->wn[longitud] != '\\textbackslash{}0')\{
longitud++;
\}

a->longitud = longitud;
\}

void liberarCadena(cadena a)\{
if(a != NULL)\{
free(a->wn);
free(a);
\}
\}

automata crearAutomata(int num\_estados, int num\_simbolos, int
estado\_inicial,
int* estados\_finales, int num\_finales, int** transiciones)\{

```

```

if(num\_estados<=0 || num\_simbolos<=0 || estado\_inicial<0 ||
    num\_finales<0 || transiciones==NULL)\{
return NULL;
\}

automata a = (automata) malloc(sizeof(Automata));
if(a == NULL)\{
return NULL;
\}

a->num\_estados = num\_estados;
a->num\_simbolos = num\_simbolos;
a->estado\_inicial = estado\_inicial;
a->num\_finales = num\_finales;

a->estados\_finales = (int*) malloc(num\_finales * sizeof(int));
if(a->estados\_finales==NULL && num\_finales>0)\{
    free(a);
return NULL;
\}
for(int i=0; i<num\_finales; i++)\{
    a->estados\_finales[i]=estados\_finales[i];
\}

a->transiciones = (int**) malloc(num\_estados * sizeof(int*));
if(a->transiciones == NULL)\{
    free(a->estados\_finales);
    free(a);
return NULL;
\}

for(int i = 0; i < num\_estados; i++) \{
    a->transiciones[i] = (int*) malloc(num\_simbolos * sizeof(int));
    if(a->transiciones[i] == NULL)\{
        for(int j=0; j<i; j++)\{
            free(a->transiciones[j]);
        \}
        free(a->transiciones);
        free(a->estados\_finales);
        free(a);
return NULL;
    \}

for(int j=0; j<num\_simbolos; j++)\{
    a->transiciones[i][j] = transiciones[i][j];
\}
\}

```

```

return a;
\}

int procesarCadena(automata a, cadena c)\{
if(a==NULL || c==NULL)\{
return 0;
\}

int estado\_actual = a->estado\_inicial;

printf(“\textbackslash{}\n **** Transiciones del automata ****\n\textbackslash{}\n\textbackslash{}\n”);
for(int i=0; i<c->longitud; i++)\{
char ch = c->wn[i];
int simbolo = ch - 'a';
if(simbolo<0 || simbolo >= a->num\_simbolos)\{
printf(“Simbolo invalido [{}c]”, simbolo+'a');
return 0;
\}

int estado\_anterior = estado\_actual;
estado\_actual = a->transiciones[estado\_actual][simbolo];

printf(“Delta(q{}d, {}c)=q{}d\textbackslash{}\n”, estado\_anterior, simbolo+'a', estado\_actual);
if(estado\_actual == -1)\{
return 0;
\}
\}

printf(“\textbackslash{}\n\textbackslash{}\n Estado final q{}d”, estado\_actual);

for(int i=0; i<a->num\_finales; i++)\{
if(a->estados\_finales[i] == estado\_actual)\{
return 1;
\}
\}

return 0;
\}

void liberarAutomata(automata a) \{
if (a == NULL) return;
for (int i = 0; i < a->num\_estados; i++) \{
free(a->transiciones[i]);
\}
free(a->transiciones);

```

```

free(a->estados\_finales);
free(a);
\}

int crearAutomatas(automata automatas[4]) \{
// Automata 1: a*ba*
int num\_estados1 = 3;
int num\_simbolos = 2; // a, b
int inicial1 = 0;
int finales1[] = \{1\};
int num\_finales1 = 1;
int transiciones1[3][2] = \{
\{0, 1\}, // q0: a->q0, b->q1
\{1, 2\}, // q1: a->q1, b->q2
\{2, 2\} // q2: a->q2, b->q2
\};
int* transiciones\_ptr1[3] = \{transiciones1[0], transiciones1
[1], transiciones1[2]\};

// Automata 2: a*bba*
int num\_estados2 = 4;
int inicial2 = 0;
int finales2[] = \{2\};
int num\_finales2 = 1;
int transiciones2[4][2] = \{
\{0, 1\}, // q0: a->q0, b->q1
\{3, 2\}, // q1: a->q3, b->q2
\{2, 3\}, // q2: a->q2, b->q3
\{3, 3\} // q3: a->q3, b->q3
\};
int* transiciones\_ptr2[4] = \{transiciones2[0], transiciones2
[1], transiciones2[2], transiciones2[3]\};

// Automata 3: a*b*
int num\_estados3 = 3;
int inicial3 = 0;
int finales3[] = \{0, 1\}; // q0 y q1 son finales
int num\_finales3 = 2;
int transiciones3[3][2] = \{
\{0, 1\}, // q0: a->q0, b->q1
\{2, 1\}, // q1: a->q2, b->q1
\{2, 2\} // q2: a->q2, b->q2
\};
int* transiciones\_ptr3[3] = \{transiciones3[0], transiciones3
[1], transiciones3[2]\};

// Automata 4: a*ba*ba*
int num\_estados4 = 4;

```

```

int inicial4 = 0;
int finales4 [] = \{2\};
int num\_finales4 = 1;
int transiciones4[4][2] = \{
\{0, 1\}, // q0: a→q0, b→q1
\{1, 2\}, // q1: a→q1, b→q2
\{2, 3\}, // q2: a→q2, b→q3
\{3, 3\} // q3: a→q3, b→q3
\};
int* transiciones\_ptr4[4] = \{transiciones4[0], transiciones4
[1], transiciones4[2], transiciones4[3]\};

automatas[0] = crearAutomata(num\_estados1, num\_simbolos,
inicial1,
finales1, num\_finales1, transiciones\_ptr1);
if (automatas[0] == NULL) return -1;

automatas[1] = crearAutomata(num\_estados2, num\_simbolos,
inicial2,
finales2, num\_finales2, transiciones\_ptr2);
if (automatas[1] == NULL) \{
liberarAutomata(automatas[0]);
return -1;
\}

automatas[2] = crearAutomata(num\_estados3, num\_simbolos,
inicial3,
finales3, num\_finales3, transiciones\_ptr3);
if (automatas[2] == NULL) \{
liberarAutomata(automatas[0]);
liberarAutomata(automatas[1]);
return -1;
\}

automatas[3] = crearAutomata(num\_estados4, num\_simbolos,
inicial4,
finales4, num\_finales4, transiciones\_ptr4);
if (automatas[3] == NULL) \{
liberarAutomata(automatas[0]);
liberarAutomata(automatas[1]);
liberarAutomata(automatas[2]);
return -1;
\}

return 0;
\}

void menuExpresiones(int opc, automata automatas[4])\{

```

```

if (opc<1 || opc>4) return;

cadena cad = crearCadena();
if (cad == NULL) \{
printf(“Error al crear la cadena.\textbackslash{}n”);
return;
\}

printf(“Ingrese la cadena a evaluar (solo a y b, para cadena
      vacia solo presiona ENTER): ”);
pedirCadena(cad);

if (procesarCadena(automatas[opc - 1], cad)) \{
printf(“\textbackslash{}tCADENA ACEPTADA\textbackslash{}n\
      \textbackslash{}n”);
\} else \{
printf(“\textbackslash{}tCADENA RECHAZADA\textbackslash{}n\
      \textbackslash{}n”);
\}

liberarCadena(cad);
\}

```

Listing 2: Implementación del archivo cabecera

```

#include<stdio.h>
#include ‘‘funciones.h’’

int main()\{
int opc;
printf(‘‘\textbackslash{}n\textbackslash{}n\textbackslash{}t
***** Practica 2 *****\textbackslash{}n’’);

automata automatas[4];
if(crearAutomatas(automatas) != 0)\{
printf(‘‘Error al crear los automatas, saliendo\textbackslash{}n
’’);
return 1;
\}

do\{
printf(‘‘\textbackslash{}n Elija la expresion regular a revisar
\textbackslash{}n\textbackslash{}n’’);
printf(‘‘1. – (a*)b(a*)\textbackslash{}n’’);
printf(‘‘2. – (a*)bb(a*)\textbackslash{}n’’);
printf(‘‘3. – a*b*\textbackslash{}n’’);
printf(‘‘4. – (a*)b(a*)b(a*)\textbackslash{}n’’);
printf(‘‘5. – Salir\textbackslash{}n\textbackslash{}n’’);

scanf(‘‘\ %d’’ , \&opc);
getchar();

if(opc>=1 \&\& opc<=4)\{
menuExpresiones(opc, automatas);
\}else if(opc != 5)\{
printf(‘‘Opcion invalida, intente de nuevo\textbackslash{}n’’);
\}
\}while(opc != 5);

for(int i=0; i<4; i++)\{
liberarAutomata(automatas[i]);
\}

return 0;
\}

```

Listing 3: Implementación del archivo cabecera

Relación entre Especificación Formal e Implementación

La implementación en C refleja fielmente la teoría de autómatas mediante los siguientes mapeos de componentes:

- **Estados (Q):** Representados por el campo `num_estados` y gestionados mediante índices enteros en la matriz de transiciones.
- **Alfabeto (Σ):** Aunque el programa acepta caracteres, estos se mapean a índices enteros mediante la operación `ch - 'a'`, restringiendo el alfabeto a $\{a, b\}$.
- **Función de Transición (δ):** Se implementa como una matriz bidimensional (`int** transiciones`), donde `transiciones[estado_actual][simbolo]` devuelve el siguiente estado.
- **Estado Inicial (q_0):** Almacenado explícitamente en el campo `estado_inicial` de la estructura `Automata`.
- **Estados Finales (F):** Un arreglo dinámico (`estados_finales`) que el programa recorre al finalizar el procesamiento de la cadena para determinar la aceptación.

El corazón del procesamiento reside en la función `procesarCadena`, que emula la función de transición δ , iterando sobre la cadena y actualizando el estado actual hasta consumir toda la entrada o alcanzar un estado de error (-1).